Quickstart

Sentence Transformer

Characteristics of Sentence Transformer (a.k.a bi-encoder) models:

1. Calculates a **fixed-size vector representation (embedding)** given **texts, images, audio, or video**.
2. Embedding calculation is often **efficient**, embedding similarity calculation is **very fast**.
3. Applicable for a **wide range of tasks**, such as semantic textual similarity, semantic search, clustering, classification, paraphrase mining, and more.
4. Often used as a **first step in a two-step retrieval process**, where a Cross-Encoder (a.k.a. reranker) model is used to re-rank the top-k results from the bi-encoder.

Once you have installed Sentence Transformers, you can easily use Sentence Transformer models:

Text | Multimodal

Documentation

1. `SentenceTransformer`
2. `SentenceTransformer.encode`
3. `SentenceTransformer.encode_query`
4. `SentenceTransformer.encode_document`
5. `SentenceTransformer.similarity`

Other useful methods and links:

- `SentenceTransformer.similarity_pairwise`
- SentenceTransformer > Usage
- SentenceTransformer > Speeding up Inference
- SentenceTransformer > Pretrained Models
- SentenceTransformer > Training Overview

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("Qwen/Qwen3-Embedding-0.6B")

query = "Which planet is known as the Red Planet?"
documents = [
    "Jupiter is the largest planet in our solar system.",
    "Mars gets its reddish color from iron minerals in its soil.",
    "Photosynthesis lets plants turn sunlight into energy.",
]

query_embedding = model.encode_query(query)
document_embeddings = model.encode_document(documents)
print(query_embedding.shape, document_embeddings.shape)
# (1024,) (3, 1024)

scores = model.similarity(query_embedding, document_embeddings)[0]
for index in scores.argsort(descending=True):
    print(f"{scores[index]:.4f}: {documents[index]}")
# 0.6115: Mars gets its reddish color from iron minerals in its soil.
# 0.4360: Jupiter is the largest planet in our solar system.
# 0.1893: Photosynthesis lets plants turn sunlight into energy.
```

- SentenceTransformer > Loss Overview
- SentenceTransformer > Training Examples

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access to private or gated models your account can read. See the login instructions.

Finetuning Sentence Transformer models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read Sentence Transformer > Speeding up Inference for tips on how to speed up inference of models by up to 2x-3x.

Cross Encoder

Characteristics of Cross Encoder (a.k.a reranker) models:

1. Calculates a **similarity score** given **pairs of inputs** (typically text, but also images or other modalities).
2. Generally provides **superior performance** compared to a Sentence Transformer (a.k.a. bi-encoder) model.

3. Often **slower** than a Sentence Transformer model, as it requires computation for each pair rather than each text.
4. Due to the previous 2 characteristics, Cross Encoders are often used to **re-rank the top-k results** from a Sentence Transformer model.

The usage for Cross Encoder (a.k.a. reranker) models is similar to Sentence Transformers:

Text | Multimodal

Documentation

1. `CrossEncoder`
2. `CrossEncoder.rank`
3. `CrossEncoder.predict`

Other useful methods and links:

- CrossEncoder > Usage
- CrossEncoder > Speeding up Inference
- CrossEncoder > Pretrained Models
- CrossEncoder > Training Overview
- CrossEncoder > Loss Overview
- CrossEncoder > Training Examples

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access to private or gated models your account can read. See the login instructions.

```python
from sentence_transformers import CrossEncoder

# 1. Load a pretrained CrossEncoder model
model = CrossEncoder("cross-encoder/stsb-distilroberta-base")

# We want to compute the similarity between the query sentence...
query = "A man is eating pasta."

# ... and all sentences in the corpus
corpus = [
    "A man is eating food.",
    "A man is eating a piece of bread.",
    "The girl is carrying a baby.",
    "A man is riding a horse.",
    "A woman is playing violin.",
    "Two men pushed carts through the woods.",
    "A man is riding a white horse on an enclosed ground.",
    "A monkey is playing drums.",
    "A cheetah is running behind its prey.",
]

# 2. We rank all sentences in the corpus for the query
ranks = model.rank(query, corpus)

# Print the scores
print("Query: ", query)
for rank in ranks:
    print(f"{rank['score']:.2f}\t{corpus[rank['corpus_id']]}")
"""
Query:  A man is eating pasta.
0.67    A man is eating food.
0.34    A man is eating a piece of bread.
0.08    A man is riding a horse.
0.07    A man is riding a white horse on an enclosed ground.
0.01    The girl is carrying a baby.
0.01    Two men pushed carts through the woods.
0.01    A monkey is playing drums.
0.01    A woman is playing violin.
0.01    A cheetah is running behind its prey.
"""

# 3. Alternatively, you can also manually compute the score between two sentences
import numpy as np

sentence_combinations = [[query, sentence] for sentence in corpus]
scores = model.predict(sentence_combinations)

# Sort the scores in decreasing order to get the corpus indices
ranked_indices = np.argsort(scores)[::-1]
print("Scores:", scores)
```

```
print("Indices:", ranked_indices)
"""
Scores: [0.6732372, 0.34102544, 0.00542465,
0.07569341, 0.00525378, 0.00536814, 0.06676237,
0.00534825, 0.00516717]
Indices: [0 1 3 6 2 5 7 4 8]
"""
```

With `CrossEncoder("cross-encoder/stsb-distilroberta-base")` we pick which CrossEncoder model we load. CrossEncoder models can also work with multimodal inputs: Qwen/Qwen3-VL-Reranker-2B can rank images and text by relevance to a query.

Finetuning CrossEncoder models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read CrossEncoder > Speeding up Inference for tips on how to speed up inference of models by up to 2x-3x.

Sparse Encoder

Characteristics of Sparse Encoder models:

1. Calculates **sparse vector representations** where most dimensions are zero.
2. Provides **efficiency benefits** for large-scale retrieval systems due to the sparse nature of embeddings.
3. Often **more interpretable** than dense embeddings, with non-zero dimensions corresponding to specific tokens.
4. **Complementary to dense embeddings**, enabling hybrid search systems that combine the strengths of both approaches.

The usage for Sparse Encoder models follows a similar pattern to Sentence Transformers:

Documentation

1. `SparseEncoder`
2. `SparseEncoder.encode`
3. `SparseEncoder.similarity`
4. `SparseEncoder.sparsity`

```python
from sentence_transformers import SparseEncoder

# 1. Load a pretrained SparseEncoder model
model = SparseEncoder("naver/splade-cocondenser-ensembledistil")

# The sentences to encode
sentences = [
    "The weather is lovely today.",
    "It's so sunny outside!",
    "He drove to the stadium.",
]

# 2. Calculate sparse embeddings by calling model.encode()
embeddings = model.encode(sentences)
print(embeddings.shape)
# [3, 30522] - sparse representation with vocabulary size dimensions

# 3. Calculate the embedding similarities (using dot product by default)
similarities = model.similarity(embeddings, embeddings)
print(similarities)
# tensor([[   35.629,     9.154,     0.098],
#         [    9.154,    27.478,     0.019],
#         [    0.098,     0.019,    29.553]])

# 4. Check sparsity statistics
stats = SparseEncoder.sparsity(embeddings)
print(f"Sparsity: {stats['sparsity_ratio']:.2%}")  # Typically >99% zeros
print(f"Avg non-zero dimensions per embedding: {stats['active_dims']:.2f}")
```

Other useful methods and links:

- SparseEncoder > Usage
- SparseEncoder > Speeding up Inference
- SparseEncoder > Pretrained Models
- SparseEncoder > Training Overview
- SparseEncoder > Loss Overview
- Sparse Encoder > Vector Database Integration

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access to private or gated models your account can read. See the login instructions.

With `SparseEncoder("naver/splade-cocondenser-ensembledistil")` we load a pretrained SPLADE model that generates sparse embeddings. SPLADE (SParse Lexical AnD Expansion) models use MLM prediction mechanisms to create sparse representations that are particularly effective for information retrieval tasks.

Finetuning Sparse Encoder models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read Sparse Encoder > Speeding up Inference for tips on how to speed up inference of models by up to 2x-3x.

Multi-Vector Encoder

Characteristics of Multi-Vector Encoder (a.k.a ColBERT-style or “late-interaction”) models:

1. Calculates a **sequence of token-level vectors** per input rather than a single fixed-size embedding.
2. Scores queries against documents with the **MaxSim** operator: for each query token, take the maximum similarity to any document token, then sum across query tokens.
3. Preserves **token-level matching information** that single-vector models discard, typically yielding stronger retrieval at the cost of a larger index footprint.
4. Recent VLM-backed variants (ColPali, ColQwen2, ColModernVBert, …) extend this to **image documents** (each image patch becomes a “token”), enabling end-to-end OCR-free document retrieval.

The usage for Multi-Vector Encoder models follows a similar pattern to Sentence Transformers:

Text | Multimodal

```python
from sentence_transformers import MultiVectorEncoder

# 1. Load a pretrained MultiVectorEncoder model
model = MultiVectorEncoder("lightonai/LateOn")

queries = ["What is the capital of France?"]
documents = [
    "Paris is the capital of France.",
    "Berlin is the capital of Germany.",
]

# 2. Encode queries and documents (note the
asymmetric encode_query / encode_document split)
query_embeddings = model.encode_query(queries)
document_embeddings =
model.encode_document(documents)

# Each entry is a 2D tensor of shape (num_tokens_i,
embedding_dim), variable-length per input.
print(query_embeddings[0].shape)
# torch.Size([10, 128])

# 3. Score with MaxSim
scores = model.similarity(query_embeddings,
document_embeddings)
print(scores)
# tensor([[9.1129, 8.8769]], device='cuda:0')
```

Documentation

1. `MultiVectorEncoder`
2. `MultiVectorEncoder.encode_query`
3. `MultiVectorEncoder.encode_document`
4. `MultiVectorEncoder.similarity`

Other useful methods and links:

- MultiVectorEncoder > Usage
- MultiVectorEncoder > Speeding up Inference
- MultiVectorEncoder > Pretrained Models
- MultiVectorEncoder > Training Overview
- MultiVectorEncoder > Loss Overview

Tip

Log in with `hf auth login` for higher Hugging Face Hub rate limits and access

With `MultiVectorEncoder("lightonai/LateOn")` we pick a pretrained late-interaction model. Late-interaction models are particularly strong for **retrieval** tasks where token-level matching matters (named entities, exact-phrase queries, sub-document relevance). ColPali-style variants extend this to image document retrieval over scanned pages, slides, and figures.

to private or gated models your account can read. See the login instructions.

Finetuning Multi-Vector Encoder models is easy and requires only a few lines of code. For more information, see the Training Overview section.

Tip

Read Multi-Vector Encoder > Speeding up Inference for benchmarks and tips on how to speed up inference with the ONNX and OpenVINO backends.

Next Steps

Consider reading one of the following sections next:

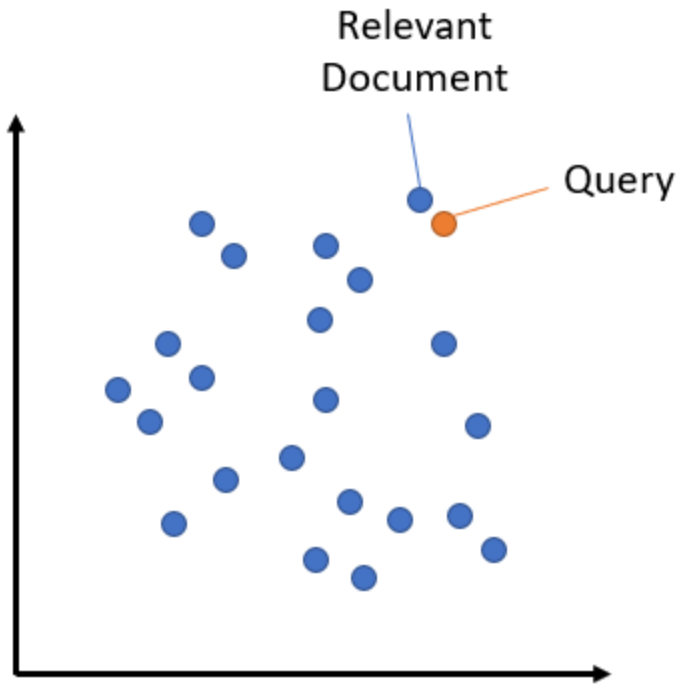
- Sentence Transformers > Usage
- Sentence Transformers > Pretrained Models
- Sentence Transformers > Training Overview
- Sentence Transformers > Training Examples > Multilingual Models
- Sentence Transformers > Training Examples > Multimodal Models
- Cross Encoder > Usage
- Cross Encoder > Pretrained Models
- Cross Encoder > Training Examples > Multimodal Models
- Sparse Encoder > Usage
- Sparse Encoder > Pretrained Models
- Sparse Encoder > Vector Database Integration
- Multi-Vector Encoder > Usage
- Multi-Vector Encoder > Pretrained Models
- Multi-Vector Encoder > Training Overview

Semantic Search

Semantic search seeks to improve search accuracy by understanding the semantic meaning of the search query and the corpus to search over. Semantic search can also perform well given synonyms, abbreviations, and misspellings, unlike keyword search engines that can only find documents based on lexical matches.

Background

The idea behind semantic search is to embed all entries in your corpus, whether they be sentences, paragraphs, or documents, into a vector space. At search time, the query is embedded into the same vector space and the closest embeddings from your corpus are found. These entries should have a high semantic similarity with the query.



Symmetric vs. Asymmetric Semantic Search

A **critical distinction** for your setup is *symmetric* vs. *asymmetric semantic search*:

- For **symmetric semantic search** your query and the entries in your corpus are of about the same length and have the same amount of content. An example would be searching for similar questions: Your query could for example be *"How to learn Python online?"* and you

want to find an entry like *"How to learn Python on the web?"*. For symmetric tasks, you could potentially flip the query and the entries in your corpus.
    - Related training example: Quora Duplicate Questions.
    - Suitable models: Pre-Trained Sentence Embedding Models
- For **asymmetric semantic search**, you usually have a **short query** (like a question or some keywords) and you want to find a longer paragraph answering the query. An example would be a query like *"What is Python"* and you want to find the paragraph *"Python is an interpreted, high-level and general-purpose programming language. Python's design philosophy ..."*. For asymmetric tasks, flipping the query and the entries in your corpus usually does not make sense.
    - Related training example: MS MARCO
    - Suitable models: Pre-Trained MS MARCO Models

It is critical **that you choose the right model** for your type of task.

> **Tip**
>
> For asymmetric semantic search, you are recommended to use `SentenceTransformer.encode_query` to encode your queries and `SentenceTransformer.encode_document` to encode your corpus.
>
> The more general `SentenceTransformer.encode` method differs in two ways from `SentenceTransformer.encode_query` and `SentenceTransformer.encode_document`:
>
> 1. If no `prompt_name` or `prompt` is provided, it uses a predefined "query" or "document" prompt, if specified in the model's `prompts` dictionary.
> 2. It sets the `task` to "document". If the model has a `Router` module, it will use the "query" or "document" task type to route the input through the appropriate submodules.
>
> Note that `SentenceTransformer.encode` is the most general method and can be used for any task, including Information Retrieval, and that if the model was not trained with predefined prompts and/or task types, then all three methods will return identical embeddings.

Manual Implementation

For small corpora (up to about 1 million entries), we can perform semantic search with a manual implementation by computing the embeddings for the corpus with `SentenceTransformer.encode_document` as well as for our query with `SentenceTransformer.encode_query`, and then calculating the semantic textual similarity using `SentenceTransformer.similarity`.

For a simple example, see semantic_search.py:

```
"""
This is a simple application for sentence embeddings: semantic search

We have a corpus with various sentences. Then, for a given query sentence,
we want to find the most similar sentence in this corpus.

This script outputs for various queries the top 5 most similar sentences in the corpus.
"""

import torch

from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# Corpus with example documents
corpus = [
    "Machine learning is a field of study that gives computers the ability to learn without being explicitly programmed.",
    "Deep learning is part of a broader family of machine learning methods based on artificial neural networks with representation learning.",
    "Neural networks are computing systems vaguely inspired by the biological neural networks that constitute animal brains.",
    "Mars rovers are robotic vehicles designed to travel on the surface of Mars to collect data and perform experiments.",
    "The James Webb Space Telescope is the largest optical telescope in space, designed to conduct infrared astronomy.",
    "SpaceX's Starship is designed to be a fully reusable transportation system capable of carrying humans to Mars and beyond.",
    "Global warming is the long-term heating of Earth's climate system observed since the pre-industrial period due to human activities.",
    "Renewable energy sources include solar, wind, hydro, and geothermal power that naturally replenish over time.",
    "Carbon capture technologies aim to collect CO2 emissions before they enter the atmosphere and store them underground.",
]
# Use "convert_to_tensor=True" to keep the tensors on GPU (if available)
corpus_embeddings = embedder.encode_document(corpus, convert_to_tensor=True)

# Query sentences:
queries = [
    "How do artificial neural networks work?",
    "What technology is used for modern space
```

Output

```
exploration?",
    "How can we address climate change challenges?",
]

# Find the closest 5 sentences of the corpus for each query sentence based on cosine similarity
top_k = min(5, len(corpus))
for query in queries:
    query_embedding = embedder.encode_query(query, convert_to_tensor=True)

    # We use cosine-similarity and torch.topk to find the highest 5 scores
    similarity_scores = embedder.similarity(query_embedding, corpus_embeddings)[0]
    scores, indices = torch.topk(similarity_scores, k=top_k)

    print("\nQuery:", query)
    print("Top 5 most similar sentences in corpus:")

    for score, idx in zip(scores, indices):
        print(f"(Score: {score:.4f})", corpus[idx])

    """
    # Alternatively, we can also use semantic_search to perform cosine similarity + topk
    from sentence_transformers.util import semantic_search
    hits = semantic_search(query_embedding, corpus_embeddings, top_k=5)
    hits = hits[0]      # Get the hits for the first query
    for hit in hits:
        print(corpus[hit['corpus_id']], "(Score: {:.4f})".format(hit['score']))
    """
```

```
Query: How do artificial neural networks work?
Top 5 most similar sentences in corpus:
(Score: 0.5926) Neural networks are computing systems vaguely inspired by the biological neural networks that constitute animal brains.
(Score: 0.5288) Deep learning is part of a broader family of machine learning methods based on artificial neural networks with representation learning.
(Score: 0.4647) Machine learning is a field of study that gives computers the ability to learn without being explicitly programmed.
(Score: 0.1381) Mars rovers are robotic vehicles designed to travel on the surface of Mars to collect data and perform experiments.
(Score: 0.0912) Carbon capture technologies aim to collect CO2 emissions before they enter the atmosphere and store them underground.

Query: What technology is used for modern space exploration?
Top 5 most similar sentences in corpus:
(Score: 0.3754) Mars rovers are robotic vehicles designed to travel on the surface of Mars to collect data and perform experiments.
(Score: 0.3669) SpaceX's Starship is designed to be a fully reusable transportation system capable of carrying humans to Mars and beyond.
(Score: 0.3452) The James Webb Space Telescope is the largest optical telescope in space, designed to conduct infrared astronomy.
(Score: 0.2625) Renewable energy sources include solar, wind, hydro, and geothermal power that naturally replenish over time.
(Score: 0.2275) Carbon capture technologies aim to
```

Optimized Implementation

Instead of implementing semantic search by yourself, you can use the `semantic_search()` function.

The function accepts the following parameters:

```
collect CO2 emissions before
they enter the atmosphere and
store them underground.

Query: How can we address
climate change challenges?
Top 5 most similar sentences
in corpus:
(Score: 0.3760) Global
warming is the long-term
heating of Earth's climate
system observed since the
pre-industrial period due to
human activities.
(Score: 0.3144) Carbon
capture technologies aim to
collect CO2 emissions before
they enter the atmosphere and
store them underground.
(Score: 0.2948) Renewable
energy sources include solar,
wind, hydro, and geothermal
power that naturally
replenish over time.
(Score: 0.0420) Machine
learning is a field of study
that gives computers the
ability to learn without
being explicitly programmed.
(Score: 0.0411) Deep learning
is part of a broader family
of machine learning methods
based on artificial neural
networks with representation
learning.
```

sentence_transformers.util.semantic_search(*query_embeddings: ~torch.Tensor, corpus_embeddings: ~torch.Tensor, query_chunk_size: int = 100, corpus_chunk_size: int = 500000, top_k: int = 10, score_function: ~collections.abc.Callable[[~torch.Tensor, ~torch.Tensor], ~torch.Tensor] = <function cos_sim>*) → list[list[dict[str, int | float]]] [source]

This function performs by default a cosine similarity search between a list of query embeddings and a list of corpus embeddings. It can be used for Information Retrieval / Semantic Search for corpora up to about 1 Million entries.

Parameters:

- **query_embeddings** (`Tensor`) – A 2 dimensional tensor with the query embeddings. Can be a sparse COO tensor.
- **corpus_embeddings** (`Tensor`) – A 2 dimensional tensor with the corpus embeddings. Can be a sparse COO tensor.

- **query_chunk_size** (*int*, *optional*) – Process 100 queries simultaneously. Increasing that value increases the speed, but requires more memory. Defaults to 100.
- **corpus_chunk_size** (*int*, *optional*) – Scans the corpus 500k entries at a time. Increasing that value increases the speed, but requires more memory. Defaults to 500000.
- **top_k** (*int*, *optional*) – Retrieve top k matching entries. Defaults to 10.
- **score_function** (Callable[[`Tensor`, `Tensor`], `Tensor`], optional) – Function for computing scores. By default, cosine similarity.

Returns: A list with one entry for each query. Each entry is a list of dictionaries with the keys 'corpus_id' and 'score', sorted by decreasing cosine similarity scores.

Return type: List[List[Dict[str, Union[int, float]]]]

By default, up to 100 queries are processed in parallel. Further, the corpus is chunked into set of up to 500k entries. You can increase `query_chunk_size` and `corpus_chunk_size`, which leads to increased speed for large corpora, but also increases the memory requirement.

Speed Optimization

To get the optimal speed for the `semantic_search()` method, it is advisable to have the `query_embeddings` as well as the `corpus_embeddings` on the same GPU-device. This significantly boost the performance. Further, we can normalize the corpus embeddings so that each corpus embeddings is of length 1. In that case, we can use dot-product for computing scores.

```
from sentence_transformers.util import dot_score, normalize_embeddings, semantic_search

corpus_embeddings = corpus_embeddings.to("cuda")
corpus_embeddings = normalize_embeddings(corpus_embeddings)

query_embeddings = query_embeddings.to("cuda")
query_embeddings = normalize_embeddings(query_embeddings)
hits = semantic_search(query_embeddings, corpus_embeddings, score_function=dot_score)
```

Elasticsearch

Elasticsearch has the possibility to index dense vectors and to use them for document scoring. We can easily index embedding vectors, store other data alongside our vectors and, most importantly, efficiently retrieve relevant entries using approximate nearest neighbor search (HNSW, see also below) on the embeddings.

For further details, see semantic_search_quora_elasticsearch.py.

OpenSearch

OpenSearch is a community-driven, open-source search engine that supports vector search capabilities. It allows you to index dense vectors and perform efficient similarity search using approximate nearest neighbor algorithms. OpenSearch can be used to implement both traditional keyword-based search (BM25) and semantic search, making it possible to compare and combine both approaches.

For an example implementation, see semantic_search_nq_opensearch.py, which shows how to use OpenSearch with the Natural Questions dataset, demonstrating both semantic search and BM25 search capabilities.

Approximate Nearest Neighbor

Searching a large corpus with millions of embeddings can be time-consuming if exact nearest neighbor search is used (like it is used by `semantic_search()`).

In that case, Approximate Nearest Neighbor (ANN) can be helpful. Here, the data is partitioned into smaller fractions of similar embeddings. This index can be searched efficiently and the embeddings with the highest similarity (the nearest neighbors) can be retrieved within milliseconds, even if you have millions of vectors. However, the results are not necessarily exact. It is possible that some vectors with high similarity will be missed.

For all ANN methods, there are usually one or more parameters to tune that determine the recall-speed trade-off. If you want the highest speed, you have a high chance of missing hits. If you want high recall, the search speed decreases.

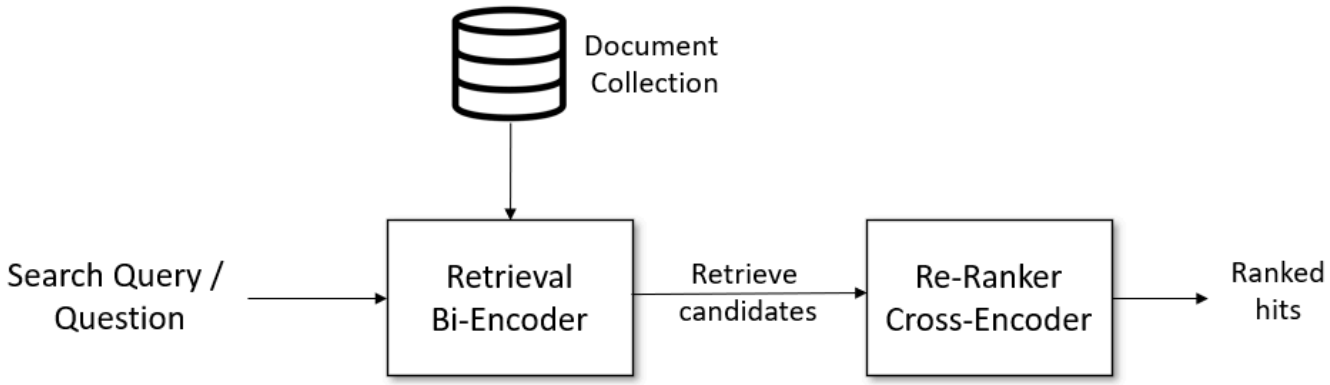
Three popular libraries for approximate nearest neighbor are Annoy, FAISS, and hnswlib.

Examples:

- semantic_search_quora_hnswlib.py
- semantic_search_quora_annoy.py
- semantic_search_quora_faiss.py

Retrieve & Re-Rank

For complex semantic search scenarios, a two-stage retrieve & re-rank pipeline is advisable:



For further details, see Retrieve & Re-rank.

Examples

We list a handful of common use cases:

Similar Questions Retrieval

semantic_search_quora_pytorch.py [Colab version] shows an example based on the Quora duplicate questions dataset. The user can enter a question, and the code retrieves the most similar questions from the dataset using `semantic_search`. As model, we use distilbert-multilingual-nli-stsb-quora-ranking, which was trained to identify similar questions and supports 50+ languages. Hence, the user can input the question in any of the 50+ languages. This is a **symmetric search task**, as the search queries have the same length and content as the questions in the corpus.

Similar Publication Retrieval

semantic_search_publications.py [Colab version] shows an example how to find similar scientific publications. As corpus, we use all publications that have been presented at the EMNLP 2016 - 2018 conferences. As search query, we input the title and abstract of more recent publications and find related publications from our corpus. We use the SPECTER model. This is a **symmetric search task**, as the paper in the corpus consists of title & abstract and we search for title & abstract.

Question & Answer Retrieval

semantic_search_wikipedia_qa.py [Colab Version]: This example uses a model that was trained on the Natural Questions dataset. It consists of about 100k real Google search queries, together with an annotated passage from Wikipedia that provides the answer. It is an example of an **asymmetric search task**. As corpus, we use the smaller Simple English Wikipedia so that it fits easily into memory.

retrieve_rerank_simple_wikipedia.ipynb [Colab Version]: This script uses the Retrieve & Re-rank strategy and is an example for an **asymmetric search task**. We split all Wikipedia articles into paragraphs and encode them with a bi-encoder. If a new query / question is entered, it is encoded by the same bi-encoder and the paragraphs with the highest cosine-similarity are retrieved. Next, the retrieved candidates are scored by a Cross-Encoder re-ranker and the 5 passages with the highest score from the Cross-Encoder are presented to the user. We use models that were trained on the MS Marco Passage Reranking dataset, a dataset with about 500k real queries from Bing search.